ORDER MANAGEMENT SYSTEM

Database Design, Sample Data, Order History & Sales Analysis

MySQL Project Documentation
Customer • Product • Orders • Order_Details

1. Project Overview

The Order Management System stores customer details, product information, orders, and the products included in each order. Primary keys uniquely identify records, while foreign keys connect customers to orders and orders/products to order details. The SQL script also demonstrates INSERT, UPDATE, DELETE, joins, grouping, and sales-analysis queries.

Project objectives

• Maintain structured customer and product records.
• Track order dates, totals, and order status.
• Record product quantities and prices for each order.
• Generate customer order history and product-demand reports.
• Calculate customer spending, average order value, and total sales.

2. Table Design & Constraints

The database contains four related tables. The following definitions summarize the fields and integrity rules in the supplied SQL script.

Customer

Column	Data type	Constraint / purpose
Customer_ID	INT	Primary key
Customer_Name	VARCHAR(50)	NOT NULL
Email	VARCHAR(100)	UNIQUE
City	VARCHAR(50)	Customer city

Product

Column	Data type	Constraint / purpose
Product_ID	INT	Primary key
Product_Name	VARCHAR(50)	NOT NULL
Price	DECIMAL(10,2)	NOT NULL; CHECK (Price > 0)

Orders

Column	Data type	Constraint / purpose
Order_ID	INT	Primary key
Customer_ID	INT	NOT NULL; FK to Customer
Order_Date	DATE	NOT NULL
Total_Amount	DECIMAL(10,2)	DEFAULT 0; CHECK (Total_Amount >= 0)
Order_Status	VARCHAR(20)	DEFAULT 'Pending'; allowed: Pending, Shipped, Delivered, Cancelled

Order_Details

Column	Data type	Constraint / purpose
Order_Detail_ID	INT	Primary key
Order_ID	INT	NOT NULL; FK to Orders
Product_ID	INT	NOT NULL; FK to Product
Quantity	INT	NOT NULL; CHECK (Quantity > 0)
Price	DECIMAL(10,2)	NOT NULL; CHECK (Price > 0)
Combined rule	—	UNIQUE (Order_ID, Product_ID)

3. Updated ER Diagram

Relationship summary

Parent table	Relationship	Child table
Customer (Customer_ID)	1 to many	Orders (Customer_ID FK)
Orders (Order_ID)	1 to many	Order_Details (Order_ID FK)
Product (Product_ID)	1 to many	Order_Details (Product_ID FK)

CUSTOMER PK Customer_ID	**ORDERS** PK Order_ID FK Customer_ID	**ORDER_DETAILS** PK Order_Detail_ID FK Order_ID, Product_ID	**PRODUCT** PK Product_ID
1	1	many	1

A customer may place multiple orders. Each order may contain multiple products through Order_Details, and a product may appear in multiple orders. The UNIQUE (Order_ID, Product_ID) constraint prevents the same product from being entered twice for one order.

4. Sample Data Used

Customer records

ID	Customer	Email	City
1	bhavesh	bhavesh@gmail.com	Chennai
2	albert	albert@gmail.com	Delhi
3	thiraviyam	thiraviyam@gmail.com	Mumbai
4	ajees	ajees@gmail.com	Chennai

Product records

ID	Product	Price (■)
101	Laptop	50,000.00
102	Mouse	500.00
103	Keyboard	1,000.00
104	Headphone	1,500.00

5. Sample Orders & Order Details

Orders after the UPDATE statements

Order ID	Customer ID	Date	Total (■)	Status
1001	1	2026-10-20	51,500.00	Shipped
1002	2	2026-10-21	2,000.00	Delivered
1003	1	2026-10-22	1,500.00	Delivered
1004	3	2026-10-23	500.00	Pending

Order_Details after the UPDATE and DELETE statements

Detail ID	Order ID	Product ID	Quantity	Price (■)
1	1001	101	1	50,000.00
2	1001	102	3	500.00
3	1002	103	2	1,000.00
4	1003	104	1	1,500.00

Note: Order 1004 remains in Orders because its status is Pending, so the DELETE condition requiring Cancelled does not match. Its Order_Details row is deleted by the earlier DELETE statement. Order 1001's total is recalculated as 1 × ■50,000 + 3 × ■500 = ■51,500.

6. CRUD Operations Demonstrated

Operation	Example from the SQL script	Purpose
Create	INSERT INTO Customer / Product / Orders / Order_Details	Adds sample records.
Read	SELECT * FROM Orders; and report queries	Displays records and analysis.
Update	Changes order statuses, quantity, and order total	Modifies existing data.
Delete	Deletes Order_Details for Order_ID 1004	Removes matching line-item data.

7. Customer Order History & Product Analysis

Order line-item report

The query joins Orders, Customer, Order_Details, and Product. It displays each order's customer, product, quantity, unit price, line subtotal, date, stored order total, and status. Because it uses INNER JOIN, order 1004 does not appear after its detail row has been deleted.

Order	Customer	Product	Qty	Subtotal (■)	Status
1001	bhavesh	Laptop	1	50,000.00	Shipped
1001	bhavesh	Mouse	3	1,500.00	Shipped
1002	albert	Keyboard	2	2,000.00	Delivered
1003	bhavesh	Headphone	1	1,500.00	Delivered

Product order-count analysis

Product	Times ordered
Laptop	1
Mouse	1
Keyboard	1
Headphone	1

Total quantity sold by product

Product	Total quantity
Laptop	1
Mouse	3
Keyboard	2
Headphone	1

The product queries use LEFT JOIN so that products without matching order details would still appear, with zero quantity where COALESCE is used. In the current data, every product appears in one order-detail record.

8. Customer Spending & Sales Metrics

Customers with the highest number of orders

Customer ID	Customer	Order count
1	bhavesh	2

The maximum-order query returns bhavesh because customer 1 has two orders; the other customers each have one order.

Customer spending (excluding cancelled orders)

Customer	Total spending (■)
bhavesh	53,000.00
albert	2,000.00
thiraviyam	500.00
ajees	0.00

The spending query includes all customers through LEFT JOIN. The amounts are based on Orders.Total_Amount and exclude orders whose status is Cancelled. Order 1004 contributes ■500 because it is still Pending, even though its order-detail row was deleted.

Sales summary

Metric	Calculated value	Calculation basis
Average order value	■13,875.00	AVG(Total_Amount), excluding cancelled orders
Total sales	■55,500.00	SUM(Total_Amount), excluding cancelled orders

Order status counts

Order status	Number of orders
Shipped	1
Delivered	2
Pending	1
Cancelled	0

These values reflect the supplied SQL statements in execution order, including the status updates, quantity update, total recalculation, and deletion of order-detail record(s).

9. Important SQL Queries

Customer order history

```
SELECT c.Customer_ID, c.Customer_Name, o.Order_ID,
       o.Order_Date, p.Product_Name, od.Quantity,
       od.Price, o.Order_Status
FROM Customer c
JOIN Orders o ON c.Customer_ID = o.Customer_ID
JOIN Order_Details od ON o.Order_ID = od.Order_ID
JOIN Product p ON od.Product_ID = p.Product_ID
ORDER BY c.Customer_ID, o.Order_Date;
```

Product quantity sold

```
SELECT p.Product_Name,
       COALESCE(SUM(od.Quantity), 0) AS Total_Quantity_Sold
FROM Product p
LEFT JOIN Order_Details od ON p.Product_ID = od.Product_ID
GROUP BY p.Product_ID, p.Product_Name;
```

Customer spending

```
SELECT c.Customer_Name,
       COALESCE(SUM(o.Total_Amount), 0) AS Total_Spending
FROM Customer c
LEFT JOIN Orders o ON c.Customer_ID = o.Customer_ID
                  AND o.Order_Status <> 'Cancelled'
GROUP BY c.Customer_ID, c.Customer_Name;
```

Sales metrics

```
SELECT AVG(Total_Amount) AS Average_Order_Value
FROM Orders
WHERE Order_Status <> 'Cancelled';

SELECT SUM(Total_Amount) AS Total_Sales
FROM Orders
WHERE Order_Status <> 'Cancelled';

SELECT Order_Status, COUNT(*) AS Total_Orders
FROM Orders
GROUP BY Order_Status;
```

10. Conclusion

This database provides a structured foundation for managing customers, products, orders, and order line items. Primary and foreign keys support record identity and relationships, while UNIQUE and CHECK constraints help protect data quality. The reporting queries demonstrate customer order history, product demand, spending totals, average order value, and overall sales.

Implementation note: The calculations in this document follow the SQL script as provided. Since Order 1004's details are deleted while its order remains Pending, reports based on order totals and reports based on line items will not show identical order sets.